

Construct Recon3D metabolic reaction network

This tutorial shows how to use python to construct a metabolic reaction network (MRN) from the Recon3D genome scale metabolic model (GEM).

Import python dependencies

```
import pandas as pd
import cobra
import networkx as nx
from collections import defaultdict
from collections import Counter
import pathlib
import os
import numpy as np
```

Set up output directory

```
out_dir = "tutorial_output/construct_recon3d_mrn"
pathlib.Path(out_dir).mkdir(parents=True, exist_ok=True)
```

Read Recon3D GEM

```
met_model = cobra.io.load_matlab_model(
    "tutorial_data/Recon3D_301.mat")

assert (
    len(met_model.reactions)
    == len(set([x.id for x in met_model.reactions]))
)

assert (
    len(met_model.metabolites)
```

```

    == len(set([x.id for x in met_model.metabolites]))
)

```

Create a dataframe of MRN nodes

```

rxn_gene_df = []
for i_rxn in met_model.reactions:
    for j in i_rxn.genes:
        rxn_gene_df.append(
            (i_rxn.id, i_rxn.name, j.id,
             str(i_rxn), i_rxn.bounds[0] < 0,
             i_rxn.bounds[1] > 0, i_rxn.subsystem))

del i_rxn, j

rxn_gene_df = pd.DataFrame(
    rxn_gene_df,
    columns=["rxn", "rxn_name", "gene_id",
             "equation", "rxn_dir_left",
             "rxn_dir_right", "pathway"])

assert not (rxn_gene_df["gene_id"] == "").any()
assert not rxn_gene_df["gene_id"].isnull().any()

assert not (rxn_gene_df["pathway"] == "").any()
assert not rxn_gene_df["pathway"].isnull().any()

# Map Entrez Gene ID to gene symbols.
#
# The mappings were obtained using BioMart.
eid_gs_df = pd.read_table(
    os.path.join(
        "tutorial_data",
        "biomart_ensembl_110_eid_hs_mapping.tsv"),
    dtype=str)

assert not eid_gs_df["gene_entrez_id"].duplicated().any()

```

```

eid_gs_dict = dict(zip(
    eid_gs_df["gene_entrez_id"], eid_gs_df["gene"]))

def extract_eid_pfx(eid):
    dot_pos = eid.find(".")
    if dot_pos >= 0:
        ep = eid[:dot_pos]
    else:
        # not found
        ep = eid
    #
    return ep

mm_gid_gsb_dict = {}
for i in rxn_gene_df["gene_id"].tolist():
    i_gs = eid_gs_dict.get(extract_eid_pfx(i), "")
    #
    if i in mm_gid_gsb_dict:
        assert mm_gid_gsb_dict[i] == i_gs
    else:
        mm_gid_gsb_dict[i] = i_gs

del i

m_rxn_gene_df = rxn_gene_df.copy()
m_rxn_gene_df["gene"] = [
    mm_gid_gsb_dict[x]
    for x in m_rxn_gene_df["gene_id"].tolist()
]

m_rxn_gene_df = m_rxn_gene_df.loc[
    ~m_rxn_gene_df["gene"].str.fullmatch(""), :]

m_rxn_gene_df = m_rxn_gene_df.loc[
    :, ["rxn", "rxn_name", "equation", "pathway", "gene"]]

def drop_dup_sort_rows(df):
    df = df.copy()
    df = df.drop_duplicates()
    df = df.sort_values(by=df.columns.tolist())

```

```

    return df

m_rxn_gene_df = drop_dup_sort_rows(m_rxn_gene_df)

```

Count the frequencies of metabolites in reactions

```

assert all([
    x.id[-3] == "[" for x in met_model.metabolites])
assert all([
    x.id[-1] == "]" for x in met_model.metabolites])
assert all([
    x.id[-2] == x.compartment
    for x in met_model.metabolites])

rxn_met_list = []
for i in met_model.reactions:
    for j in i.metabolites:
        rxn_met_list.append(j.id)
        assert j.compartment == j.id[-2]

del i, j

met_counter = Counter([x for x in rxn_met_list])

compartment_counter = Counter(
    [x[-2] for x in rxn_met_list])

f_mets = set([
    "h", "h2o", "atp", "pi", "adp", "na1", "nadp", "nadph",
    "coa",
    "o2", "hco3", "crn", "ppi", "nad", "amp", "nadh", "co2",
    "udp",
    "h2o2", "nh4", "cl", "accoa", "ppcoa", "fad", "fadh2",
    "occoa",
    "amp", "chsterol", "so4",
])

```

```

met_weight_dict = {}
for k, v in met_counter.items():
    if k[:3] in f_mets:
        met_weight_dict[k] = 0
    else:
        met_weight_dict[k] = compartment_counter[k[-2]] / v

del k, v

assert min(list(met_weight_dict.values())) >= 0

```

Create a dataframe of MRN edges

```

edge_dict = {}
for i_rxn in met_model.reactions:
    i_mets = set([x.id for x in i_rxn.metabolites])
    for j_rxn in met_model.reactions:
        j_mets = [x.id for x in j_rxn.metabolites]
        # weight between i and j, so init 0
        weight = 0
        for k in j_mets:
            if k in i_mets:
                weight += met_weight_dict[k]
        #
        # undirected graph
        edge_key = tuple(sorted([i_rxn.id, j_rxn.id]))
        if edge_key not in edge_dict:
            edge_dict[edge_key] = weight
        else:
            assert weight == edge_dict[edge_key]

del i_rxn, j_rxn, i_mets, j_mets, weight, k, edge_key

edge_df = pd.DataFrame(
    [(k[0], k[1], v) for k, v in edge_dict.items()],
    columns=["src", "dest", "mn_weight"]
)

```

```

assert (
    set(edge_df["src"].tolist())
    == set(edge_df["dest"].tolist())
)
assert not edge_df.duplicated().any()
assert not edge_df.duplicated(subset=["src", "dest"]).any()

```

Output MRN node and edge dataframes

```

m_rxn_gene_df.to_csv(
    os.path.join(
        out_dir,
        "recon3d_unfiltered_nodes.tsv"),
    sep="\t", index=False)

# Only output pruned edges to save file hosting space
edge_weights = edge_df["mn_weight"].values

edge_weight_threshold = np.round(
    np.percentile(
        edge_weights[edge_weights > 0],
        20),
    3)

pruned_edge_df = edge_df.loc[
    edge_df["mn_weight"] > edge_weight_threshold, :]

pruned_edge_df.to_csv(
    os.path.join(
        out_dir,
        "recon3d_pruned_edges.tsv.gz"),
    sep="\t", index=False,
    compression={
        "method": "gzip",
        "compresslevel": 1,
        "mtime": 1})

```

■ End of Document ■